

Every project starts the same way.

Every asset is a script.

One install gives every Roblox project the same docs, rules and pipelines: `/setup` writes the standard into the project, headless Blender builds the assets, offline renders replace Studio screenshots, and scripts live on disk so several Claude agents can build one game at once.

4 days Joust Tycoon, from design talk to an approved gameplay MVP	14 skills in the install, one of them new: <code>/studio</code>	19 hard rules seeded into every project's tech design	≈4 s to build an asset and render its four-view sheet
---	---	---	---

VIEW · `/SETUP`

What one `/setup` gives a project

Run `/setup MyGame` with the place open in Studio. It asks what the game is, what it should look like and how animation should feel, then writes all of this. Run it on an existing project and it audits the docs and the live place instead, and offers fixes without overwriting anything.

LAYER	WHAT LANDS IN THE PROJECT	WHAT IT STOPS
Docs skeleton	<code>index.md</code> , <code>tech-design.md</code> , <code>updateLog.md</code> , <code>features/</code>	Every session re-reading the codebase to find its footing. Sessions read two files, then only the feature doc they need.
Hard rules	19 rules pasted into <code>tech-design.md</code> : component reuse, the <code>Assets</code> module, <code>const</code> , Scale-only UI, rendered item pictures, systems attached to instances, component-owned state, Blender for meshes, offline checks and more	The same mistakes being learned again in every game

LAYER	WHAT LANDS IN THE PROJECT	WHAT IT STOPS
Art style	<code>## Project Art Style</code> , the palette as code in <code>blender-source/style.py</code> , an optional painter style profile	Assets from different sessions looking like different games
Animation style	<code>features/animation-style.md</code> in the user's own words, grown from every rejection	Repeating the same correction about feel
Asset pipeline	<code>blender-source/</code> and a smoke build that proves Blender works on this machine	Hand-placed vertices, and finding a broken toolchain halfway through the first asset
Code on disk	<code>studio.json</code> and <code>game-source/</code> , mirrored by <code>/studio</code>	Code that only exists in Studio, clobbered edits, no diffs
Economy	<code>features/economy.md</code> with value tags, and <code>tools/balance-sim.js</code>	Numbers scattered across configs, and pacing guesses with nothing behind them
Baseplate	<code>RunnerBaseplate.rbxl</code> : the Runner framework, AdminSystem and asset folders, with nothing from any one game. Kept as Rojo source and re-extracted from a live game by script	Every game starting from a different, half-cleaned copy of the last one
Templates	<code>CONTRACTS.md</code> for parallel agents, <code>ui-instances.md</code> , <code>ui-backlog.md</code>	Sub-agents colliding, and nobody knowing which GUI is which

The asset loop

Each asset is a Python script. Claude writes it, builds it in Blender with no window, reads the rendered sheet, fixes and rebuilds until it reads right, and only then touches Studio. Nothing is uploaded until the user approves.

1

Script

```
blender-  
source/Asset.py
```

Shapes from Blender modifiers and bmesh ops, colours by name from the palette, sockets and attributes authored in the script.

2

Build

```
blend.js build ·  
≈4 s
```

Four-view contact sheet, size in studs, triangles per part, warnings for holes, flipped faces and budget.

3

Paint

```
paint-only rebuild  
≈28 s
```

For painted styles: one texture per MeshPart, painted by library brushes, baked on the GPU and cached.

4

Preview

```
blend.js push
```

Built as MeshParts in

`Workspace.Blender Imports`. Refuses the wrong place and a running playtest.

5

Look

```
/assetshot · ≈15 s  
for 8 views
```

Rendered offline from the instances in Studio. Studio's camera never moves.

6

Approve, then upload

```
push --upload
```

Mesh and Image assets, straight into `ReplicatedStorage.Assets.Models` or the map.

7

Icon

```
/itemicon
```

A transparent, outlined render for shops and inventories, stamped on the model as `IconImage`.

Why build in Blender at all

Real modelling tools

Bevel, mirror, array, solidify, boolean, spin and inset replace hundreds of hand-placed vertices in Luau. A crate is a few dozen lines.

Judged before Studio

Every build renders a four-view sheet with geometry checks. Bad meshes are caught in seconds, before anything reaches the place.

The script is the asset

Diffable, reviewable and regenerable. It survives what Studio loses: rolled-back writes, and previews that turn to checkerboards after a restart.

A library that compounds

84 painters in two style profiles, Painted Toon and Soft Painted. Painters written for one game are promoted into the library on approval, so the next game starts richer.

Many sessions at once

Headless means no window and no shared viewport, so several agents build and render in parallel. Ten of Joust Tycoon's castle tiers were built at once, one agent per tier.

What it does not cover

Plain Parts (collision, zones, greybox), UI, VFX, animations, skinned meshes, and art that must stay real Parts for studs. Those keep their usual tools.

[VIEW](#) · [INVENTORY](#)

The skills

Sizes are lines of tool code; the rest of each skill is its written guide. Everything runs on plain Node and Python with no packages to install.

STANDARD

/setup guide + 4 templates

Creates or audits a project to the standard; ships the standard itself and the change-summary rule.

/studio NEW 696 lines

Scripts mirrored to disk with conflict-checked pushes; run any Luau file in Studio; one bridge module for new tools; the team's Studio MCP rules.

/deslopify guide

Sweeps a project's Luau against the hard rules, one category at a time, with approval.

ASSETS

/blenderassets 8,039 lines

The headless Blender pipeline: blendlib, paintlib, shapelib, inklib and the brush library.

/assetshot 2,212 lines

Offline renders of any asset, posed rig or animation frame.

/itemicon 318 lines

Transparent outlined item icons for UI, uploaded and stamped on the model.

/importmeshtools 333 lines

robloxMeshTools' EditableMesh kit, for projects that build meshes from Luau.

PUBLISHING

/create-animation

guide

Uploads a KeyframeSequence as an Animation, with no API key.

/create-devproduct · **/create-gamepass**

guide

Monetisation products through Open Cloud, registered in the project's config.

/upload-images

guide

Bulk Decal uploads with rate limiting.

WIRING

/makegui

guide

A ScreenGui wired into the Runner framework: Frame, Button and a Gui component.

/makeweapon

guide

A gun model wired into the FPS layer.

/makereactcomponent

guide

An authored GUI turned into a react-lua component.

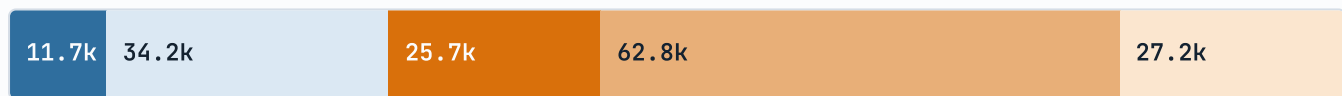
VIEW · GENERIC VS GAME

How much of a project is the setup

Measured on Joust Tycoon on 2026-10-06, in lines of code. The blue share came with the setup and the baseplate. The orange share is the game itself, written with the setup's tools and to its rules.

GENERIC · 45.9K LINES · 28%

THE GAME · 115.7K LINES · 72%



The skills (this repo)	11.7k	Runner boilerplate and libraries	34.2k
Game code: components, UI, game Shared	25.7k	Asset scripts (387 Blender files)	62.8k
Map, UI and animation build scripts	27.2k		

Lines understate the split. Every structural decision came from the setup: where things live, how they are built, checked, pushed, documented and approved. The game had to supply only what makes it Joust Tycoon.

VIEW · CASE STUDY

Joust Tycoon, day by day

Ordered by date, from the project's update log (43 entries in four days).

10-03
1 entry

- Design settled in conversation; **/setup** wrote the docs, the hard rules, the animation style and the economy.

10-04

15 entries

- Release balance and the 22-knight roster, checked in the pacing sim.
- Painted Toon chosen and approved on a starter set (lance, armour, horse). Its painted pipeline went into `/bLenderassets` for every later game.
- All seven lance and armour tiers and five horses built, each with five mutations: 114 rigged templates uploaded.
- Mounted clips generated and uploaded, shared by every horse.

10-05

13 entries

- Greybox of the whole world from one script.
- The Stone Keep castle kit, then ten more castle tiers built one sub-agent per tier, all approved and uploaded.
- Greenmeadow terrain, forest, farmland, fair and ambient life.

10-06

14 entries

- The Ashen Fields, built one sub-agent per asset family.
- Full UI in the approved Candy Maximal style, every panel built by a script.
- Gameplay MVP: five sub-agents to one contract, audited twice, approved.

VIEW · OPERATING RULES

The rules that make it work

Docs wait for approval

A change closes with a two-to-three sentence summary. The docs are written once the user says it works, so nothing wrong gets documented.

Claude never playtests

It makes the change and says what to check.
The user plays.

Never through Studio's camera

Every visual check is an offline render. The camera is shared by every session and the user.

Change the script, not the output

Maps, GUIs, assets and code all have a source on disk. Studio runs what was pushed.

One reference asset, then volume

Settle the style on one approved asset before building a batch.

Parallel agents work to a contract

Agents edit their own files and compile-check. One orchestrator pushes, uploads and rebuilds shared things.

VIEW · INSTALL

Install it

1. Clone the repo and run the installer. It checks Node, git, Claude Code and Blender, installs the skills, fetches robloxMeshTools, writes the team rules into `~/Claude/CLAUDE.md` and smoke-builds a crate.

```
powershell -ExecutionPolicy Bypass -File install.ps1 -ProjectsDir "D:\Roblox"
```

2. Install the Studio MCP plugin from its releases page into your Plugins folder, and turn on *Allow HTTP Requests* in each place.
3. Register the MCP server with Claude Code.

```
claude mcp add robloxstudio -- npx -y robloxstudio-mcp@latest
```

4. Open `baseplate/RunnerBaseplate.rbxl` in Studio, publish it as a new experience, and set the project up.

```
/setup MyGame
```

macOS and Linux use `./install.sh --projects ~/Roblox`. The JARVIS voice is opt-in with `-Persona jarvis`.

[VIEW](#) · [LIMITS](#)

Known limits

PLATFORM Tested on Windows only. The tools are plain Node and Blender, so macOS should work, but nobody has run it there yet.

GUIS Offline renders do not draw GUIs. GUI layout is checked by reading sizes through the MCP, and the final look is judged in Play.

STUDIO MCP A third-party plugin whose upstream was archived on 2026-06-06; a maintained fork exists. The tools only need its `execute_luau` bridge.

Generated from the live setup on 2026-10-06. The repo's `maintainer/export.js` regenerates the skills whenever they improve.